프론트엔드 아키텍처 개선 및 공통 컴포넌트 리팩터링 프로젝트

1. 프로젝트 개요 (Overview)

- **프로젝트**: Health Helper App (사용자 운동 기록 및 통계 시각화 서비스)
- **기술 스택**: Next.js 14, TypeScript, Tailwind CSS, Lucide React, Recharts
- **리팩터링 동기**:

  - 기능 확장에 따른 비동기 상태 처리 코드의 기하급수적 중복 발생 - 서드파티 라이브러리(Recharts)의 파편화된 사용으로 인한 유지보수성 저하 - UI 컴포넌트와 비즈니스 로직의 강한 결합으로 인한 재사용성 한계

2. 해결 과제 및 접근 방식 (Key Challenges)

2.1 비동기 상태처리의 선언적 구조화 (`AsyncStateWrapper`)

- **문제점**: `showWeek` , `showPlans` 등 각 컴포넌트에서 데이터 로딩, 에러, 빈 상태 처리를 위해 매번 동일한 `if-else` 분기 로직(약 20~30줄)을 반복 작성함.
- **해결책**: **Generic**과 **Render Props** 패턴을 활용한 `AsyncStateWrapper` 공통 컴포넌트 설계.
- **기술적 포인트**:

  - 데이터 존재 여부에 따라 `loadingSkeleton` , `errorState` , `emptyState` 를 선언적으로 주입. - Render Props를 통해 자식 컴포넌트에 타입이 추론된 데이터를 안전하게 전달.
- **성과**: 비동기 데이터 렌더링 코드의 70% 이상을 공통화하여 가독성 및 개발 효율 비약적 향상.

2.2 Recharts의 내부 메커니즘을 고려한 차트 표준화 (`BaseChart`)

- **문제점**: 차트 구성 시마다 중복되는 SVG 마크업 발생 및 커스텀 래퍼 사용 시 Recharts 내부의 `displayName` 매칭 로직으로 인해 차트가 렌더링되지 않는 버그 발생.
- **해결책**: **Compound Component 패턴**을 도입하고 원본 컴포넌트를 직접 매핑하여 라이브러리 호환성 확보.
- **기술적 포인트**:

  - `BaseChart.Bar` , `BaseChart.Tooltip` 등 계층형 인터페이스 제공으로 캡슐화 달성. - **Factory 패턴** (`createTooltipFormatter`)을 도입하여 HTML 문자열 반복 작성을 제거하고 설정 기반으로 툴팁 디자인 생성.
- **성과**: 새로운 통계 차트 추가 시 필요한 코드를 기존 대비 약(30~40%)축소.

2.3 제어의 역전을 통한 컴포넌트 범용성 확보 (`DataTable` 리팩터링)

- **문제점**: 범용 테이블 컴포넌트 내부에 특정 운동 태그 데이터나 API 호출 로직이 하드코딩되어 다른 도메인에서 재사용 불가.
- **해결책**: **Dependency Injection(의존성 주입)** 개념을 Props에 적용하여 비즈니스 로직을 외부로 격리.
- **성과**: `DataTable` 을 순수 UI 컴포넌트로 분리하여 프로젝트 내 어떤 데이터 모델과도 결합 가능한 수준으로 개선.

---

3. 개발 생산성 혁신 (AI-Augmented Engineering)

- **시스템 가이드라인 수립**: 단순 리팩터링에 그치지 않고, 프로젝트 루트에 `.cursorrules` 가 가이드라인을 작성.
- **내용**: "코드 2회 반복 시 추출", "관심사 비대화 및 로직 혼재 시 서브컴포넌트화" 등의 구체적인 **Refactoring Triggers** 명시.
- **결과**: AI 에이전트와 협업 시에도 설계 원칙이 일관되게 적용되는 **자동화된 품질 관리 체계** 구축.

---

4. 성과 및 배운 점 (Retrospective)

- **디자인 패턴의 실전 적용**: 이론으로만 알던 Compound, Factory, IoC 패턴을 실제 서비스의 병목 지점에 적용하여 유지보수 비용을 낮추는 성취감을 얻음.
- **문서화의 가치**: 리팩터링 과정을 기록하며 스스로의 설계 논리를 정교화하고, 팀원(또는 미래의 나)에게 명확한 가이드를 제공하는 문서화의 중요성을 체감함.